

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Repository Link: https://github.com/Atharsh2910/Deep_Learning_Lab/tree/main/Lab7

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	ATHARSH S U	
Register Number	24011101008	
Roll Number	24110049	
Class	AI-DS 'A'	

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

Setup used in this execution: the first 10,000 MNIST training images were used, of which 9,000 form the training set and 1,000 (10%) form the validation set, together with the first 2,000 MNIST test images. The resulting shapes are (9000, 28, 28, 1) for training, (1000, 28, 28, 1) for validation and (2000, 28, 28, 1) for testing; the pixel range is 0–1 and the flattened input of the FC model has shape (9000, 784). All runs used random seed 42, TensorFlow 2.20.0 with Keras 3.13.2, on a Kaggle GPU (Tesla T4 $\times$ 2).

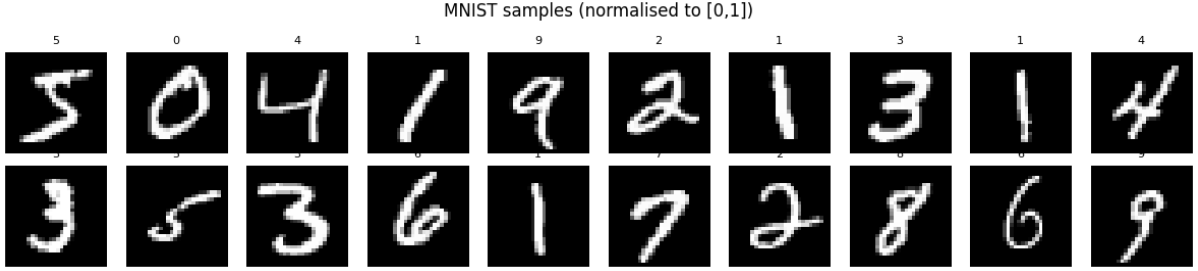


Figure 1: MNIST training samples, normalised to $[0, 1]$.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
 Reconstructed image
 Reconstruction error

For the denoising model, display:

Original clean image
 Noisy image
 Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

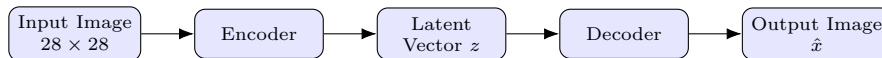
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

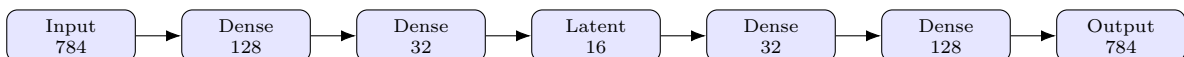
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

The encoder has 105,136 parameters, the decoder 105,904, and the complete FC autoencoder has 211,040 trainable parameters. The latent layer itself has no activation. Training time for 20 epochs was 11.0 s.

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

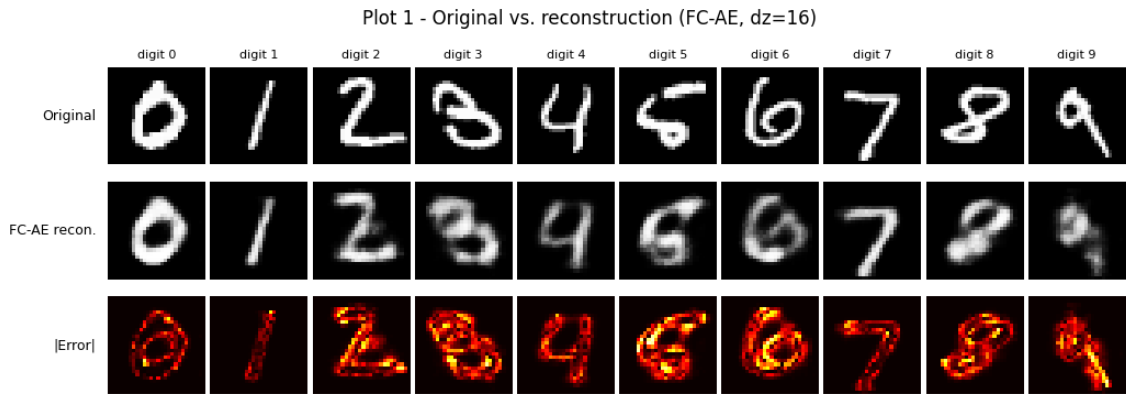


Figure 2: Plot 1: Original, FC-AE reconstruction ($d_z = 16$) and absolute-error map for ten test images.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

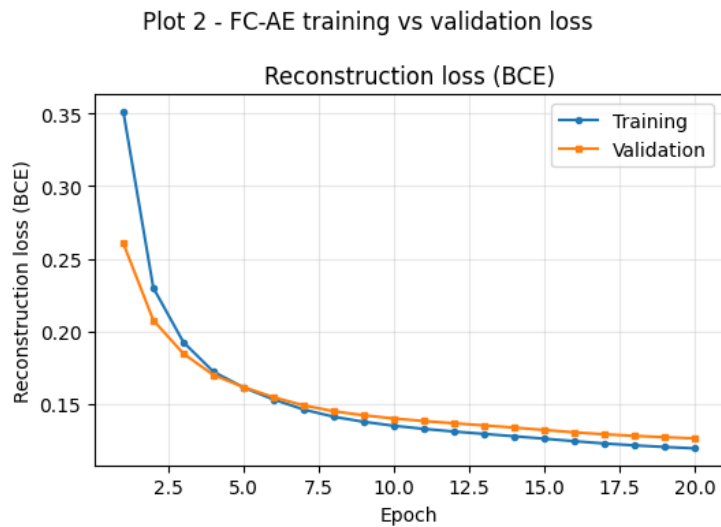


Figure 3: Plot 2: FC-AE training and validation reconstruction loss (binary cross-entropy) versus epoch.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

Report:

Metric	Value
Test MSE	0.02046
Test MAE	0.05514
Mean SSIM	0.75857

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:

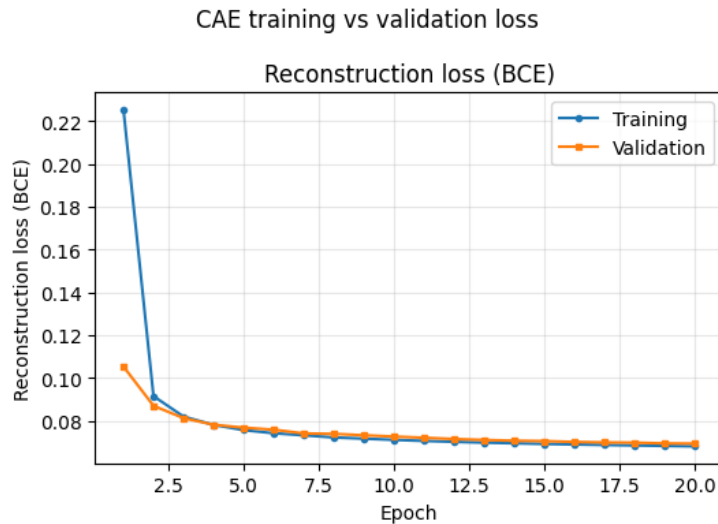
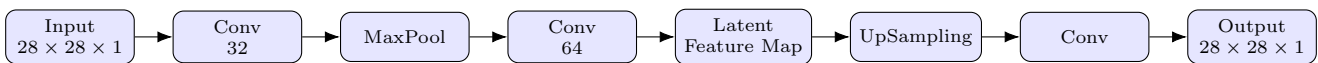


Figure 4: CAE training and validation reconstruction loss (binary cross-entropy).

Final training loss 0.0681, final validation loss 0.0694, gap 0.0013; the best validation epoch is 20/20 and the validation loss changed by 0.76% over the last three epochs, i.e. the CAE is converging/near a plateau with no clear overfitting.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02046	0.05514	0.75857	211,040
Convolutional AE	0.00259	0.01500	0.97395	74,497

Plot 3: Reconstruction Comparison

Display:

Original → FC-AE Reconstruction → CAE Reconstruction.

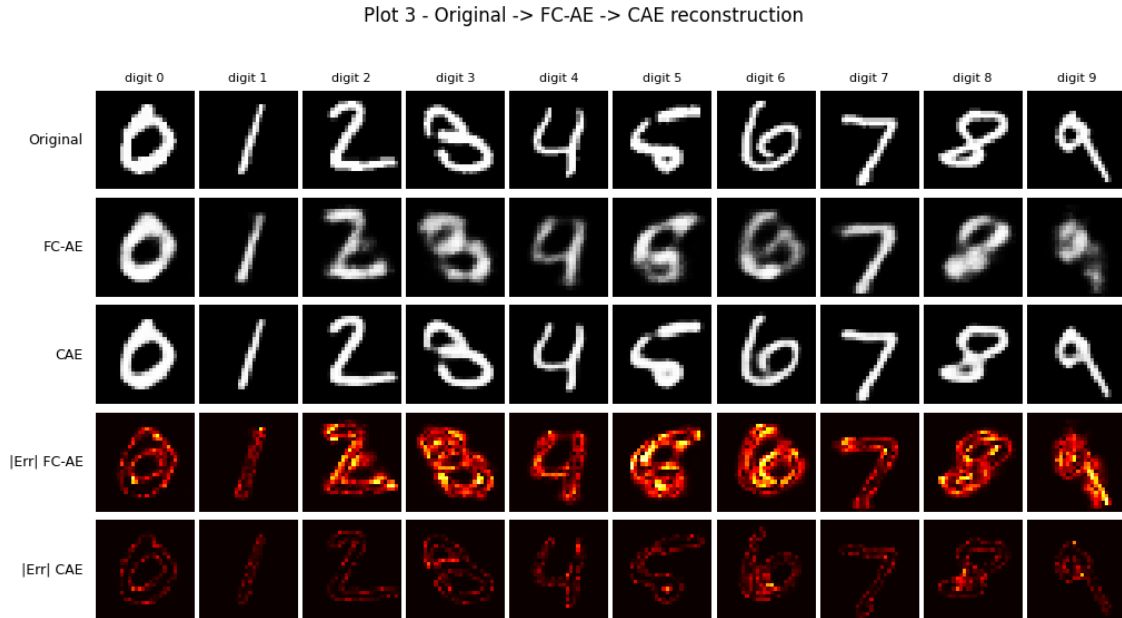


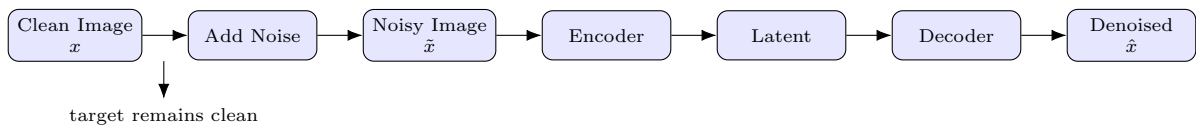
Figure 5: Plot 3: Original → FC-AE → CAE reconstructions with the corresponding absolute-error maps.

12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

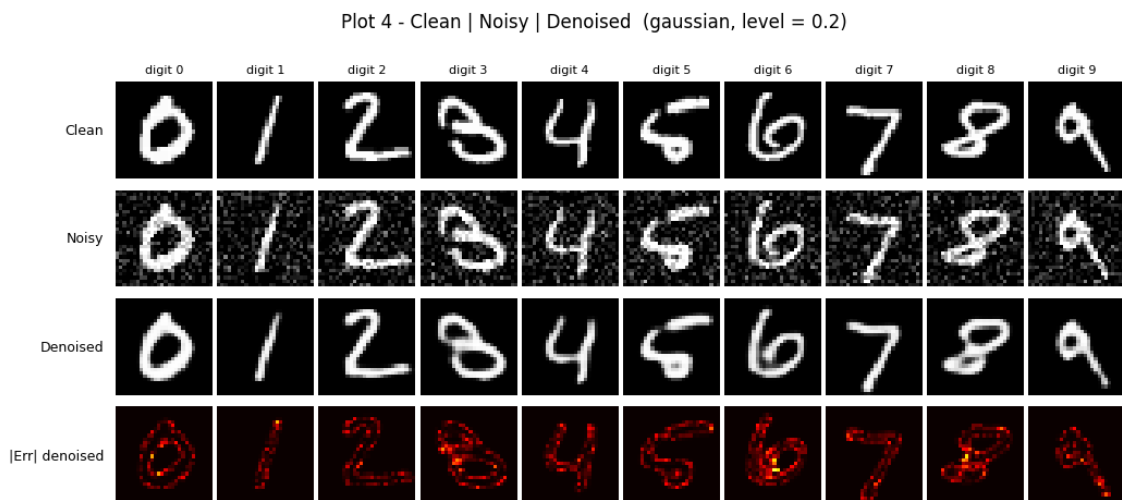


Figure 6: Plot 4: Clean | Noisy | Denoised for ten test images (Gaussian noise, $\sigma = 0.2$), with absolute error of the denoised output.

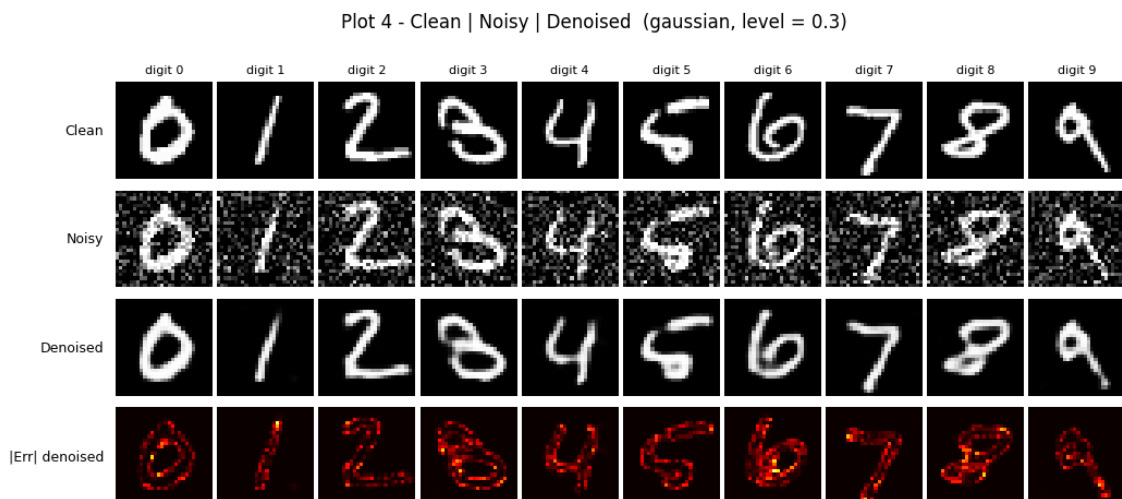


Figure 7: Plot 4 (continued): Gaussian noise, $\sigma = 0.3$.

Plot 4 - Clean | Noisy | Denoised (saltpepper, level = 0.1)

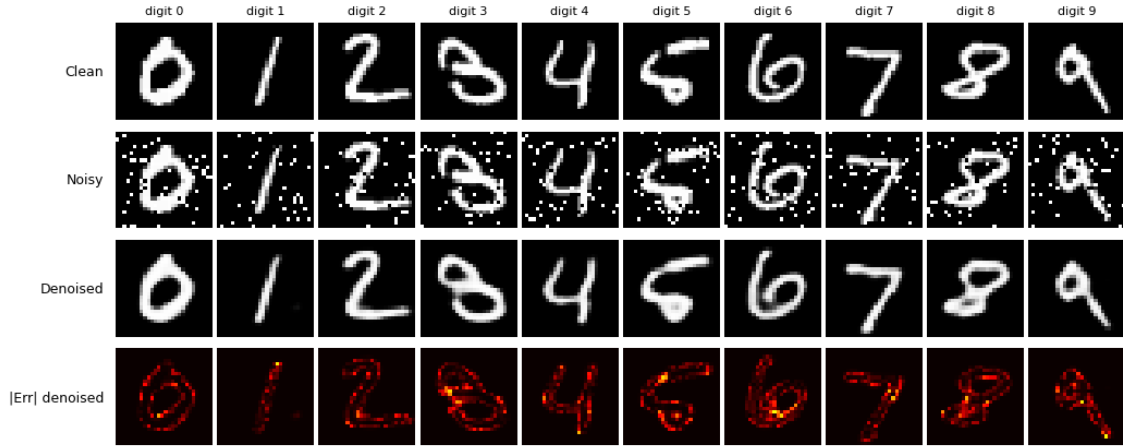


Figure 8: Plot 4 (continued): salt-and-pepper noise, $p = 0.10$.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.00340	0.01766	0.96089
0.2	0.00448	0.02091	0.94638
0.3	0.00653	0.02642	0.91534

Noise Level (p , salt-and-pepper)	MSE	MAE	SSIM
0.05	0.00423	0.01953	0.95431
0.10	0.00506	0.02164	0.94335
0.20	0.00761	0.02759	0.90526

Noisy input (before denoising)	Level	MSE	MAE	SSIM
Gaussian	0.1	0.00543	0.04408	0.67892
Gaussian	0.2	0.02122	0.08664	0.59385
Gaussian	0.3	0.04666	0.12808	0.50938
Salt-and-pepper	0.05	0.02406	0.02497	0.70834
Salt-and-pepper	0.10	0.04830	0.05014	0.57743
Salt-and-pepper	0.20	0.09585	0.09952	0.44776

Plot 5 - Noise level vs reconstruction quality (Gaussian)

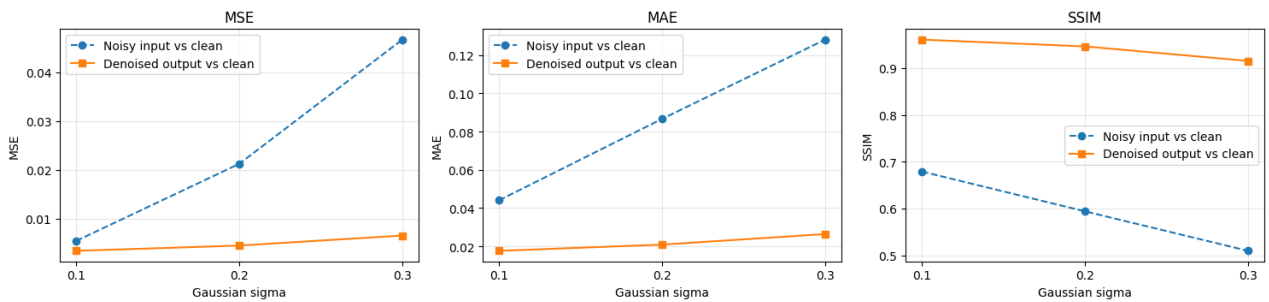


Figure 9: Plot 5: Gaussian noise level versus MSE, MAE and SSIM (noisy input and denoised output).

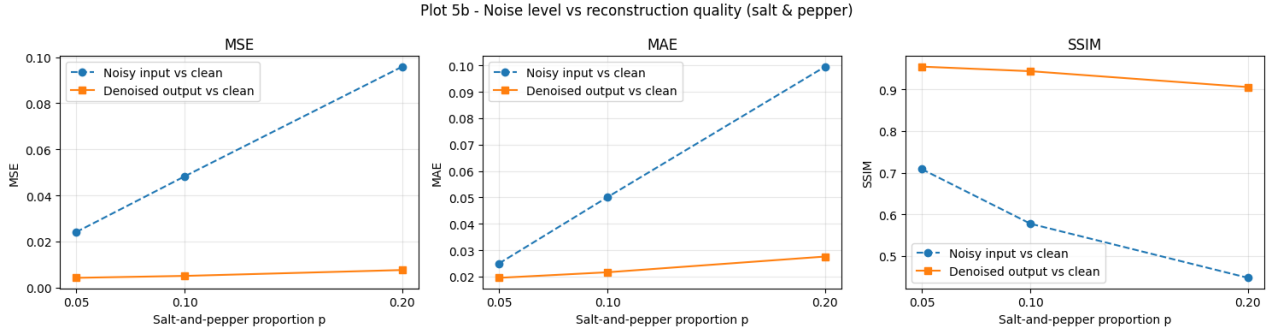


Figure 10: Plot 5 (continued): salt-and-pepper noise level versus MSE, MAE and SSIM.

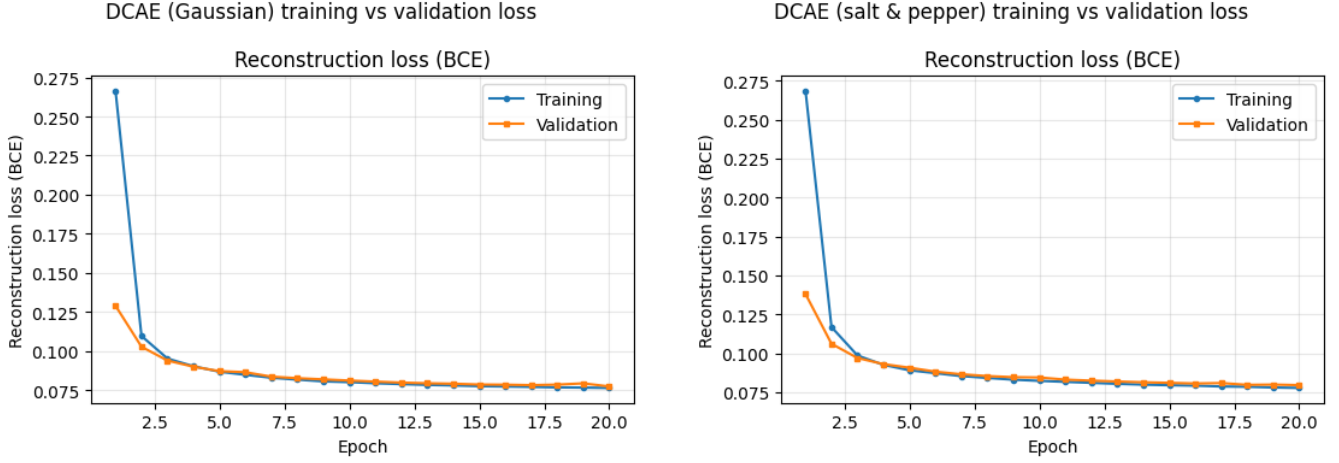


Figure 11: Training and validation loss of the DCAE trained on Gaussian noise (left) and on salt-and-pepper noise (right).

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

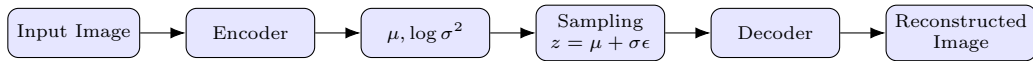
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

$$z \in \mathbb{R}^2.$$

The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

18. Exploring the VAE Latent Space

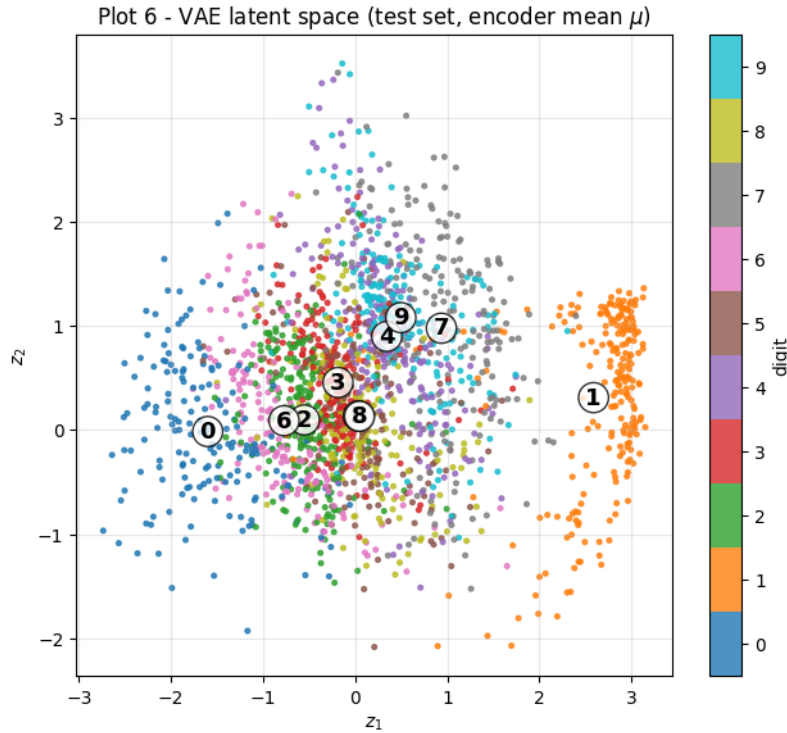


Figure 12: Plot 6: VAE latent space of the test set (encoder mean μ), coloured by digit label; circled numbers mark the class centroids.

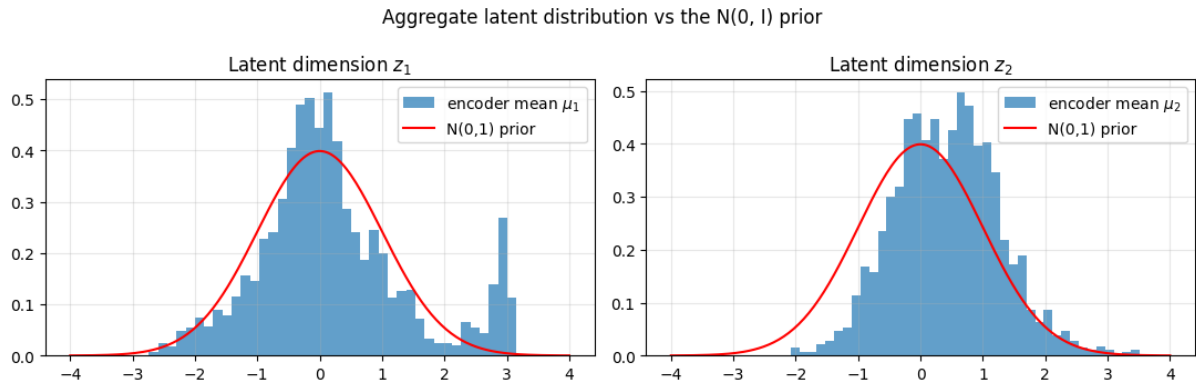


Figure 13: Marginal distribution of the encoder means μ_1 and μ_2 compared with the $\mathcal{N}(0, 1)$ prior.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Plot 7: Randomly Generated Images

Plot 7 - 25 images generated from $z \sim \mathcal{N}(0, I)$ (title = classifier prediction / confidence)



Figure 14: Plot 7: 25 images generated from $z \sim \mathcal{N}(0, I)$ (title = auxiliary-classifier prediction and confidence).

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

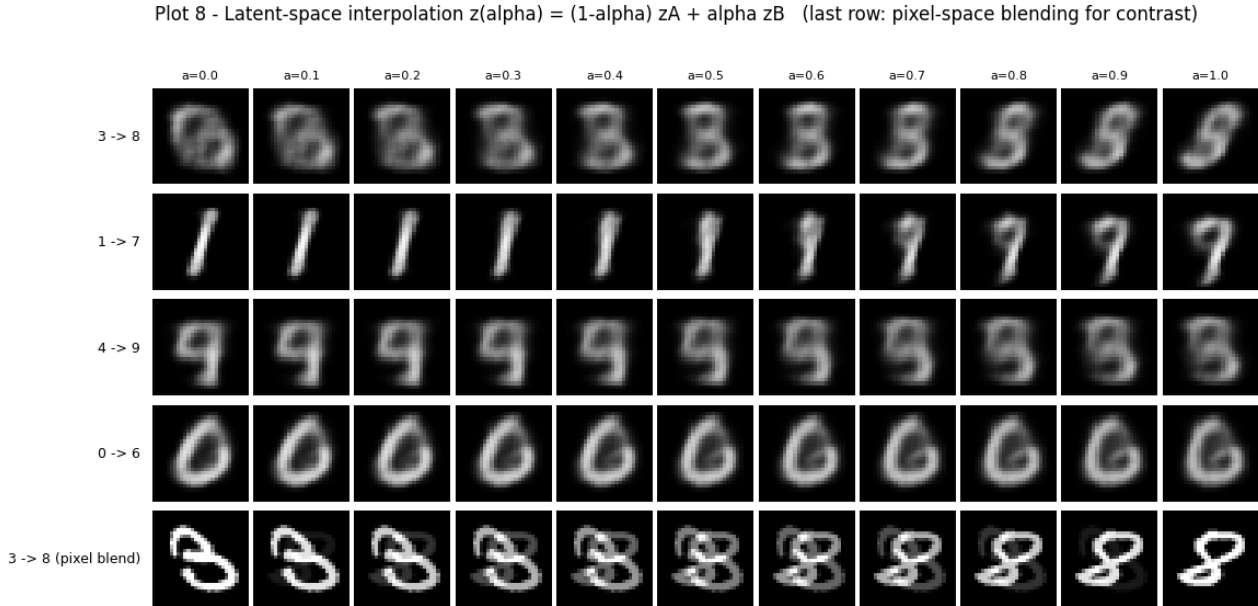


Figure 15: Plot 8: Latent-space interpolation $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$ between the codes of test digits (3→8, 1→7, 4→9, 0→6); the last row shows pixel-space blending 3→8 for contrast.

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02046	0.05514	0.75857	211,040
Convolutional AE	0.00259	0.01500	0.97395	74,497
Denoising CAE	0.00448	0.02091	0.94638	74,497

VAE Metric	Value
Reconstruction Loss	155.78459
KL Loss	5.28605
Total Loss	161.07064
Test MSE	0.04639
Test MAE	0.10963
Mean SSIM	0.44144

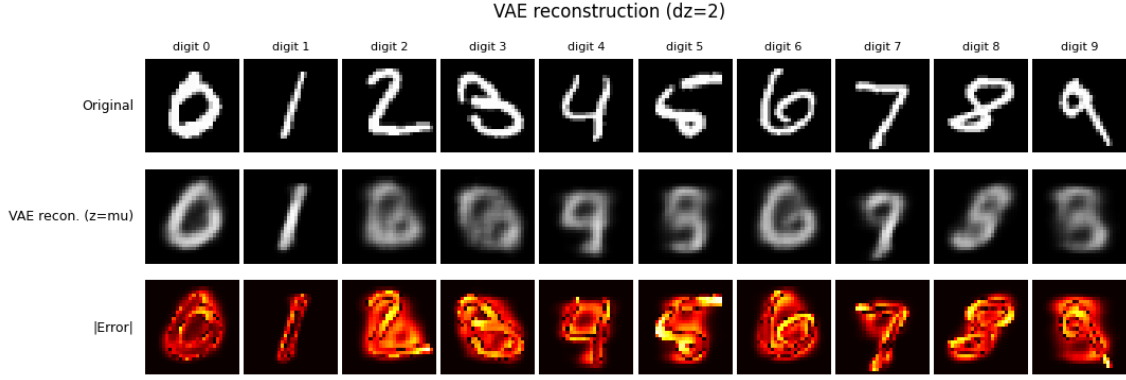


Figure 16: VAE reconstruction ($d_z = 2, z = \mu$): original, reconstruction and absolute-error map for ten test images.

22. Required Plots

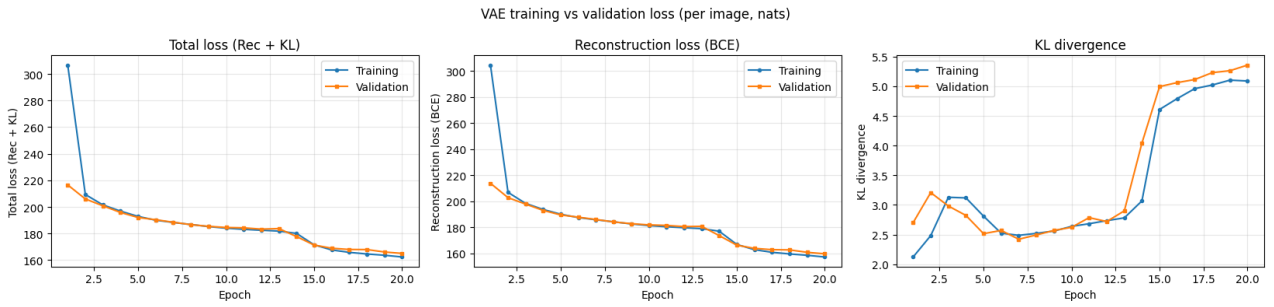


Figure 17: Plot 9 (required list, item 9): VAE training and validation total loss, reconstruction loss and KL divergence (per image, nats).

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

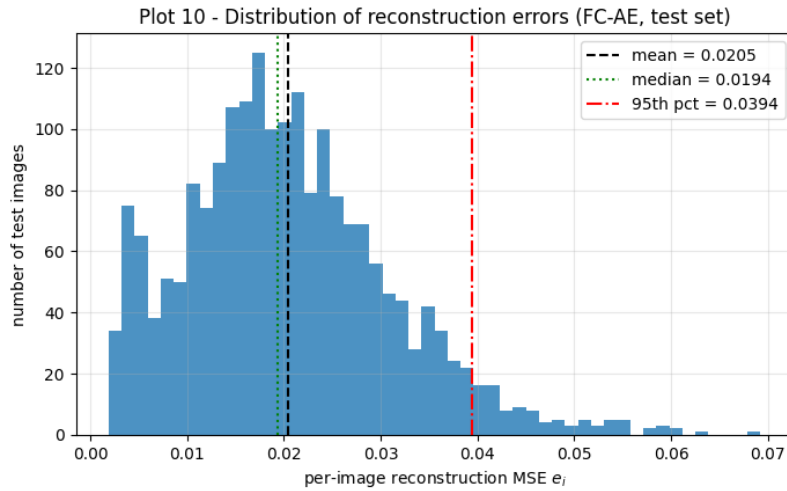


Figure 18: Plot 9 (Section 23): Distribution of the per-image reconstruction error e_i of the FC-AE over the test set (mean, median and 95th percentile marked).

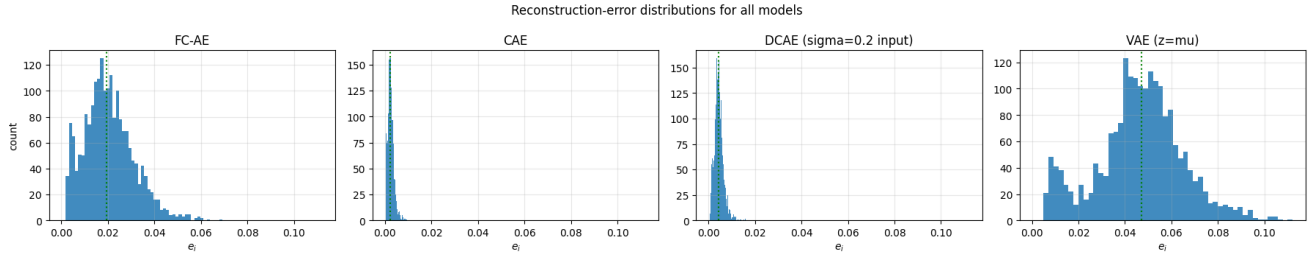


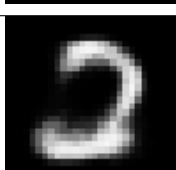
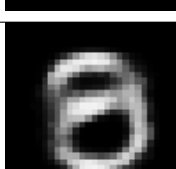
Figure 19: Distribution of e_i for all reconstruction models (FC-AE, CAE, DCAE with $\sigma = 0.2$ input, VAE with $z = \mu$).

Model	mean	median	std	5th pct	95th pct	max	skew
FC-AE	0.02046	0.01940	0.01066	0.00434	0.03944	0.06926	0.69759
CAE	0.00259	0.00238	0.00138	0.00072	0.00511	0.01092	1.38424
DCAE ($\sigma = 0.2$ input)	0.00448	0.00426	0.00199	0.00154	0.00802	0.01589	1.00582
VAE ($z = \mu$)	0.04639	0.04719	0.01851	0.01063	0.07576	0.11211	-0.04212

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

Display:

Sample	Original	Reconstruction
1		
2		
3		
4		
5		

Sample	Test index	Digit	MSE	SSIM	Ink (mean pixel)
1	1782	8	0.0693	0.4112	0.1501
2	1758	8	0.0627	0.5543	0.1810
3	1170	8	0.0610	0.4980	0.1709
4	744	2	0.0606	0.5181	0.1671
5	338	8	0.0595	0.5609	0.1846

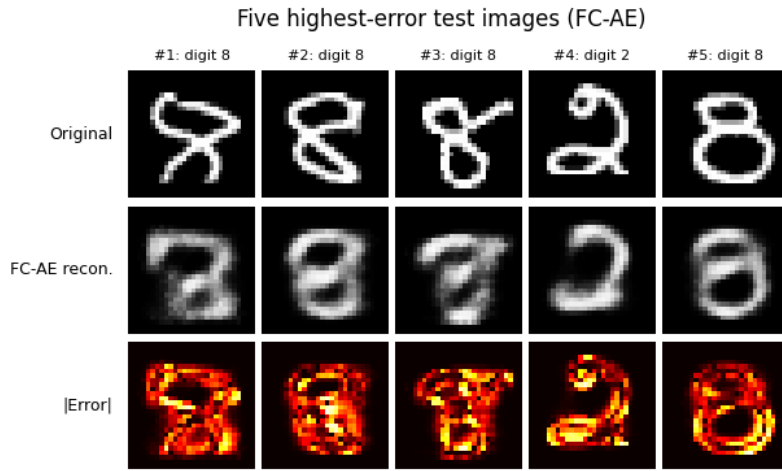


Figure 20: Five highest-error test images for the FC-AE (original, reconstruction and absolute error).

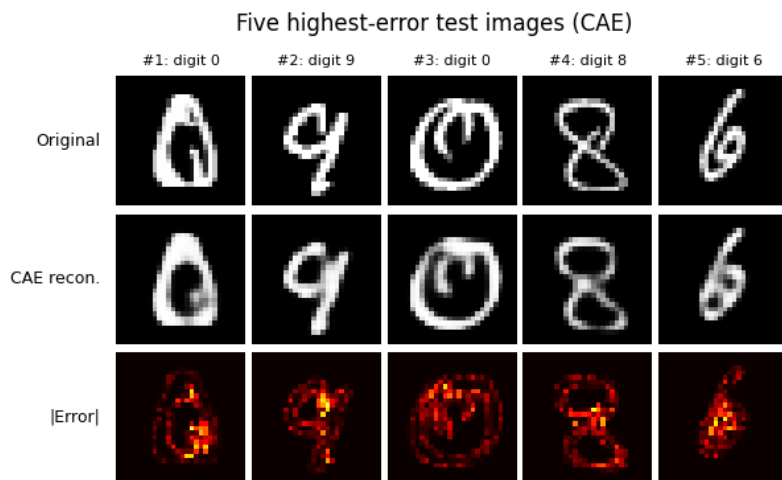


Figure 21: Five highest-error test images for the CAE.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time (s)
FC Autoencoder	0.02046	0.05514	0.75857	211,040	10.97
Conv. Autoencoder	0.00259	0.01500	0.97395	74,497	18.35
Denoising CAE	0.00448	0.02091	0.94638	74,497	18.32
VAE	0.04639	0.10963	0.44144	134,165	21.35

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn?

The following inferences are mandatory:

- effect of latent dimension on reconstruction;
- difference between fully connected and convolutional reconstruction;
- effect of Gaussian noise level;
- ability of the denoising autoencoder to recover image structure;
- characteristics of the VAE latent distribution;
- structure of the two-dimensional VAE latent space;

- quality and diversity of generated samples;
- smoothness of latent-space interpolation;
- relationship between reconstruction error and visual quality; and
- differences between deterministic and probabilistic latent representations.

Decoder fed with $z \sim \mathcal{N}(0, I)$	Mean confidence	% confident (≥ 0.9)	% ambiguous (< 0.6)
FC-AE decoder	0.629	4.75	45.75
VAE decoder	0.655	15.85	43.95

27. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.04577	0.45803	210,130
8	0.02663	0.68926	210,520
16	0.02046	0.75857	211,040
32	0.01892	0.77724	212,080

Plot 10: Latent Dimension vs. Reconstruction MSE

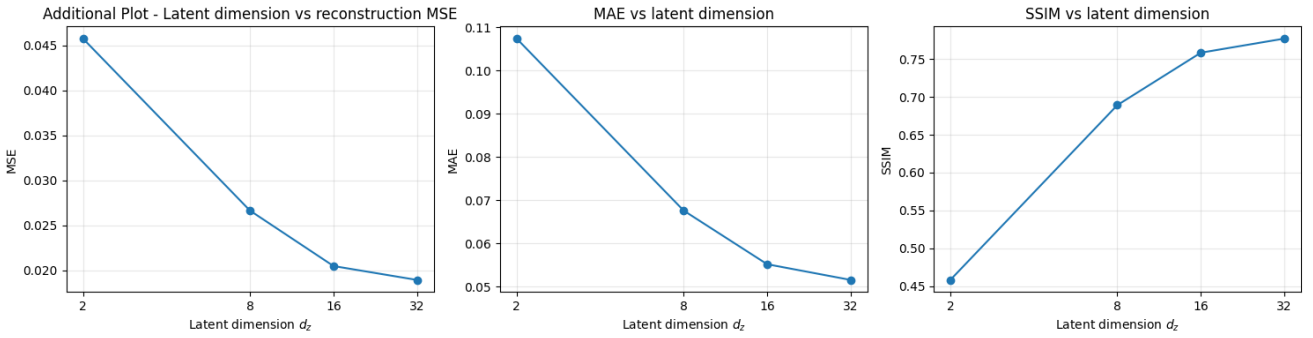


Figure 22: Plot 10: Latent dimension versus test MSE (left), MAE (centre) and SSIM (right) of the FC autoencoder.

Effect of latent dimension on FC-AE reconstruction

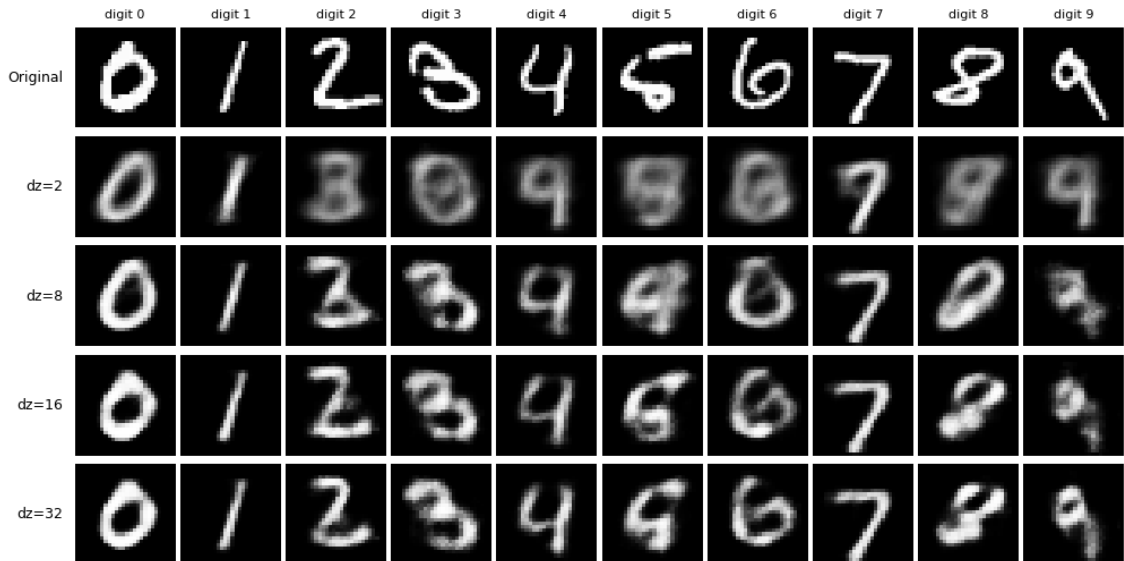


Figure 23: Effect of the latent dimension on the FC-AE reconstructions ($d_z = 2, 8, 16, 32$).

28. Discussion Questions

1. What is an autoencoder?
An autoencoder is a type of deep learning that is trained to reconstruct the original input, by encoding and decoding, with a latent space between the encoder and decoder.
2. What is the purpose of the encoder?
An encoder compresses higher dimensional data into a lower dimensional representation, feature extraction and latent space generation.
3. What is the purpose of the decoder?
A decoder uses the latent vector, expands it and reconstructs the original input.
4. What is a latent representation?
Latent representation is the lower dimensional feature representation, which is compressed by the encoder.
5. Why is the latent representation usually smaller than the input?
Latent space is generally under complete so as to extract only the most meaningful features from the input.
6. What happens when the latent dimension is too small?
A too small latent dimension may lead to loss of meaningful features from the input, leading to higher reconstruction loss and dissimilar output.
7. What happens when the latent dimension is increased?
Increase in latent dimension may lead to improvement in reconstruction loss, but a too high latent dimension may lead the autoencoder to memorize the input.
8. Why can a convolutional autoencoder preserve spatial information more effectively?
Convolutional autoencoders contains the convolution layer, that uses global average pooling and filters that helps in neighborhood information being preserved. It preserves the neighbourhood property.
9. Differentiate an autoencoder and a denoising autoencoder.
An autoencoder is used to reconstruct the original input whereas a denoising autoencoder is used to remove the noise from a noisy input and reconstruct a denoised output.
10. Why is the clean image used as the target in denoising?
The model should remove the noise and produce a clear output of the original pixel. Thus using the clean image as target helps in optimal reconstruction.
11. What happens when the noise level is increased?
Increase in noise level increases the MSE, MAE and SSIM decreases. Higher noise level thus degrades the quality of output restored image.
12. Why are MSE, MAE and SSIM useful for image reconstruction?
MSE penalizes larger errors and MAE gives the absolute deviation. SSIM measures the similarity of the output image and the original input by matching the similarity.
13. What is a Variational Autoencoder?
VAE is an generative autoencoder which uses probabilistic methods to distribute the the parameters using mean and standard deviation, to give a continual distribution. It introduces reparametrization trick to make the parameters differentiable.
14. How does a VAE differ from a conventional autoencoder?
Traditional autoencoders are used to reconstruct the original image through compression and expansion whereas VAEs are used to generate a variance in the input using probablistic methods and make the vectors continuous.
15. Why does a VAE learn a distribution instead of a single latent vector?
VAE learns the distribution to make the latent space continous, differentiable that aids in generation of variation in the input image.
16. What is the reparameterization trick?
Reparameterization trick introduces Epsilon, which is multiplied with the standard deviation and added to mean to make the vector differentiable for gradient descent.
17. Why is ϵ sampled from a standard normal distribution?
Epsilon is sampled from standard normal distribution to remove the randomness from the parameters.
18. What is the role of KL divergence in the VAE loss?
KL Divergence is used to measure the difference in distribution among the original distribution and the new generated reconstruction. KL Divergence penalizes the deviation, to minimize the reconstruction loss.
19. What is the prior distribution used in the VAE?
Standard normal distribution.
20. Why is a two-dimensional latent space useful for visualization?
A 2-D latent space is useful to infer the clustering, overlap and the structure of the latend space.
21. What does latent-space interpolation demonstrate?
Latent space interpolation demonstrates the learning of the generative model. It shows the continous distribution and generalization.
22. Why can a VAE generate new images?
VAE is capable of generating new images, as it converts the input data into a continuous probabilistic latent

space, thereby introducing variance into the input. This variance produces new images.

23. Why might some VAE-generated images be unclear?

Some VAE Generated images could be unclear due to latent space overfitting, KL Divergence loss deviation and issues with decoders.

24. What is the difference between reconstruction and generation?

Reconstruction of an image aims to perfectly recreate the original input image whereas generation introduces variations to the input and generates a new image, which may or may not have a ground truth to validate against.

25. Why should test images remain unseen during training?

Test image should be hidden during training to prevent data leakage, overfitting and to generalize the data perfectly. The unseen data helps in proper validation of the model's generalization ability.

29. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.

Model	MSE	MAE	SSIM	Parameters
CAE $d_z = 4$	0.03389	0.08068	0.63333	65,797
CAE $d_z = 8$	0.02100	0.05518	0.76492	90,889
CAE $d_z = 16$	0.01096	0.03447	0.88132	141,073
CAE $d_z = 32$	0.00621	0.02427	0.93291	241,441
CAE $7 \times 7 \times 64$ map (ref.)	0.00259	0.01500	0.97395	74,497

d_z	FC-AE MSE	CAE (dense) MSE	FC-AE SSIM	CAE (dense) SSIM
8	0.02663	0.02100	0.68926	0.76492
16	0.02046	0.01096	0.75857	0.88132
32	0.01892	0.00621	0.77724	0.93291

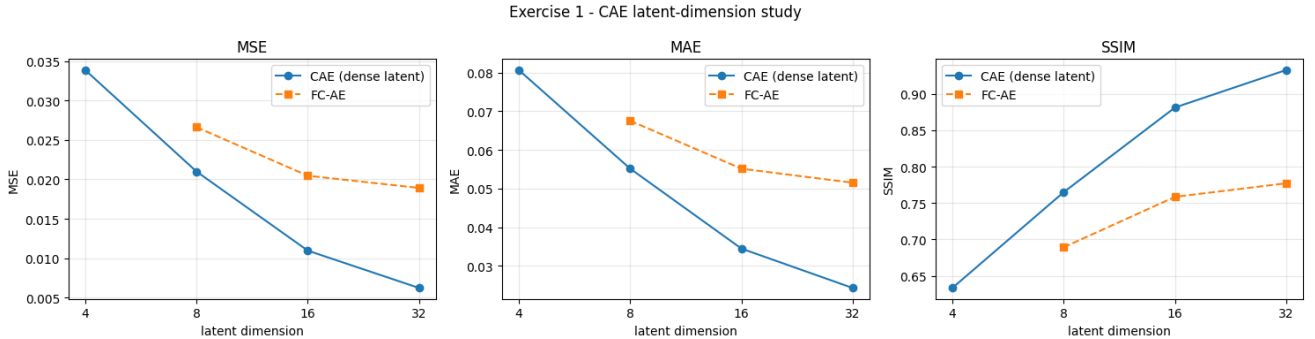


Figure 24: CAE (dense latent) versus FC-AE for latent dimensions 4–32 (MSE, MAE, SSIM).

Exercise 1 - reconstructions

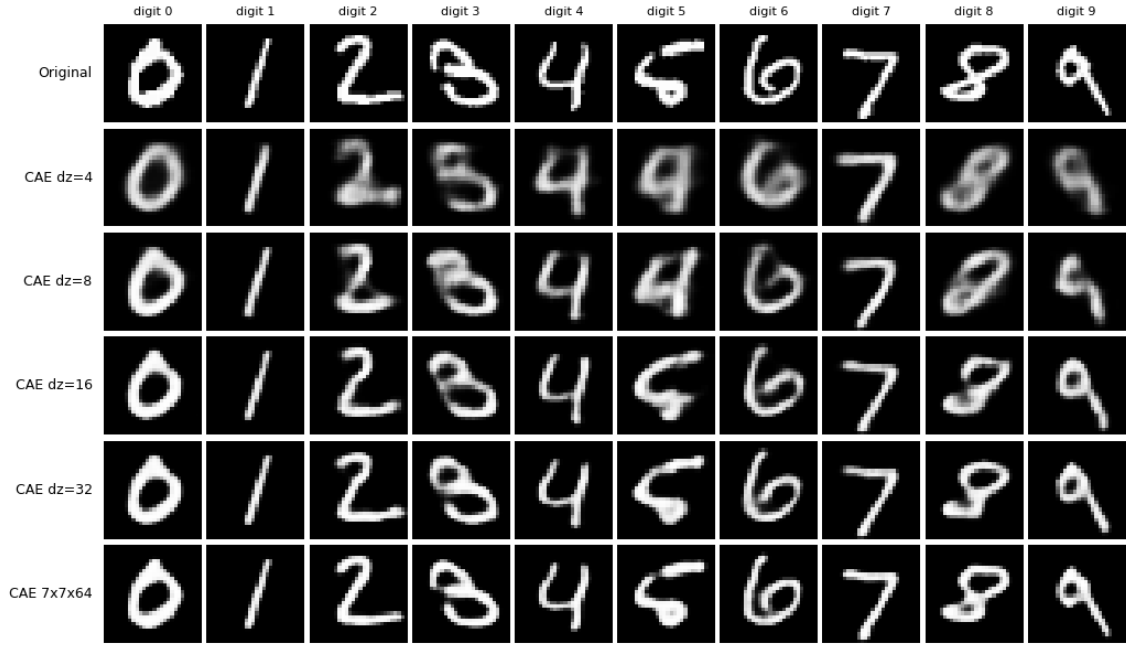


Figure 25: Exercise 1: reconstructions of the CAE for $d_z = 4, 8, 16, 32$ and of the CAE with the $7 \times 7 \times 64$ latent map.

2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.

Trained on	Test noise	Noisy SSIM	Denoised MSE	Denoised MAE	Denoised SSIM
Gaussian	Gaussian 0.1	0.6789	0.0034	0.0177	0.9609
Gaussian	Gaussian 0.2	0.5939	0.0045	0.0209	0.9464
Gaussian	Gaussian 0.3	0.5094	0.0065	0.0264	0.9153
Gaussian	Salt-and-pepper 0.05	0.7083	0.0046	0.0206	0.9405
Gaussian	Salt-and-pepper 0.1	0.5774	0.0065	0.0251	0.9104
Gaussian	Salt-and-pepper 0.2	0.4478	0.0111	0.0348	0.8386
Salt-and-pepper	Gaussian 0.1	0.6789	0.0084	0.0329	0.8515
Salt-and-pepper	Gaussian 0.2	0.5939	0.0142	0.0503	0.6954
Salt-and-pepper	Gaussian 0.3	0.5094	0.0206	0.0685	0.6073
Salt-and-pepper	Salt-and-pepper 0.05	0.7083	0.0042	0.0195	0.9543
Salt-and-pepper	Salt-and-pepper 0.1	0.5774	0.0051	0.0216	0.9433
Salt-and-pepper	Salt-and-pepper 0.2	0.4478	0.0076	0.0276	0.9053

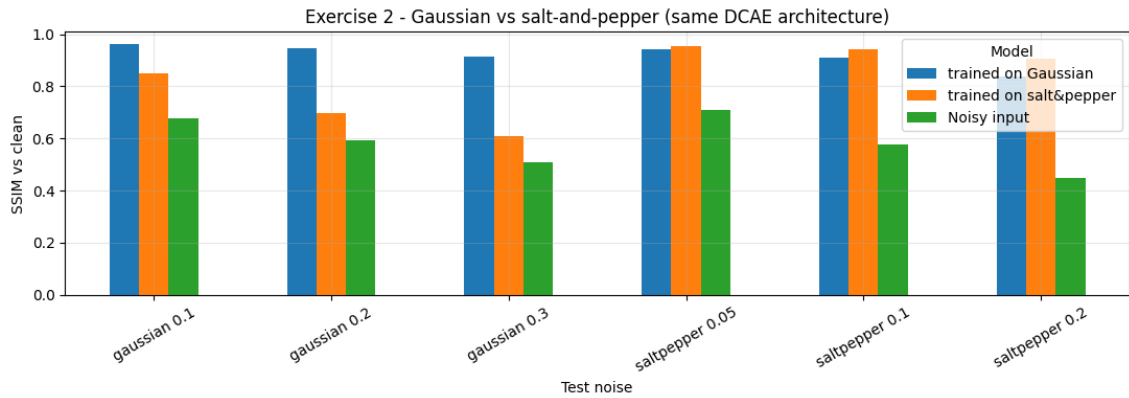


Figure 26: denoised SSIM of the two DCAEs on Gaussian and salt-and-pepper test noise (noisy-input SSIM shown for reference).

3. Train the denoising autoencoder using two different noise levels and compare SSIM.

SSIM, training noise	test $\sigma = 0.1$	test $\sigma = 0.2$	test $\sigma = 0.3$
$\sigma \in \{0.1, 0.2, 0.3\}$ (main)	0.9609	0.9464	0.9153
$\sigma = 0.1$	0.9653	0.8962	0.7117
$\sigma = 0.3$	0.9507	0.9402	0.9151

MSE, training noise	test $\sigma = 0.1$	test $\sigma = 0.2$	test $\sigma = 0.3$
$\sigma \in \{0.1, 0.2, 0.3\}$ (main)	0.00340	0.00448	0.00653
$\sigma = 0.1$	0.00307	0.00476	0.00929
$\sigma = 0.3$	0.00452	0.00519	0.00676

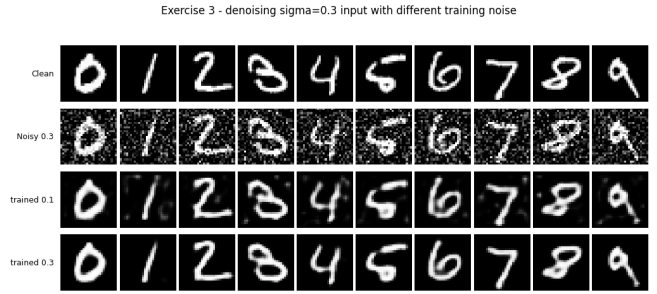
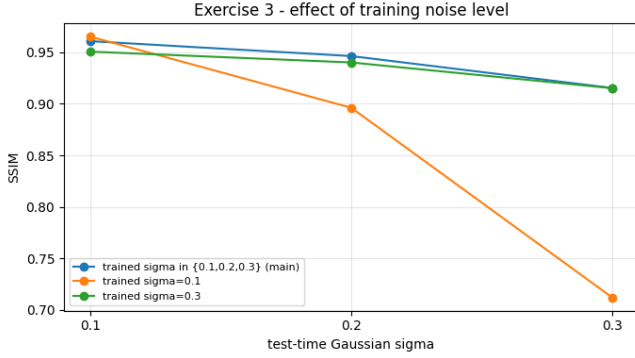


Figure 27: test SSIM versus test-time σ for different training noise levels (left) and denoising of a $\sigma = 0.3$ input (right).

- Replace the convolutional decoder with transposed convolutions and compare the reconstructions.

Model	MSE	MAE	SSIM	Parameters	Time (s)
UpSampling+Conv CAE	0.00259	0.01500	0.97395	74,497	18.35
Conv2DTranspose CAE	0.00248	0.01467	0.97436	74,497	14.60

Exercise 4 - upsampling vs transposed-convolution decoder

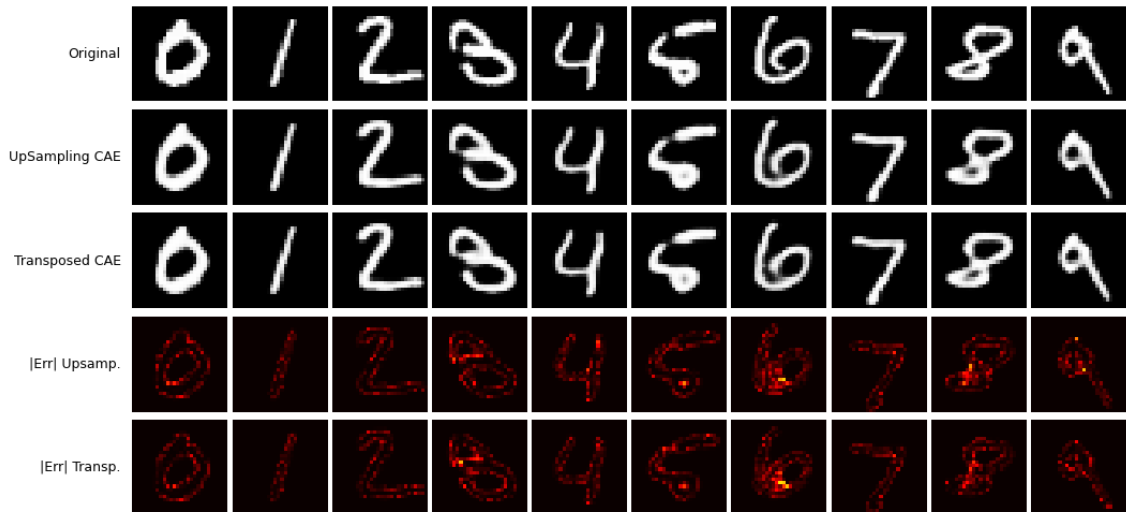


Figure 28: upsampling decoder versus transposed-convolution decoder (reconstructions and error maps).

- Implement a VAE with latent dimension 2 and 8 and compare the latent representations.

Model	Rec. loss	KL loss	Total loss	MSE	MAE	SSIM	Parameters
VAE $d_z = 2$	155.7846	5.2861	161.0706	0.0464	0.1096	0.4414	134,165
VAE $d_z = 8$	179.7217	3.0032	182.7249	0.0559	0.1304	0.3069	153,185

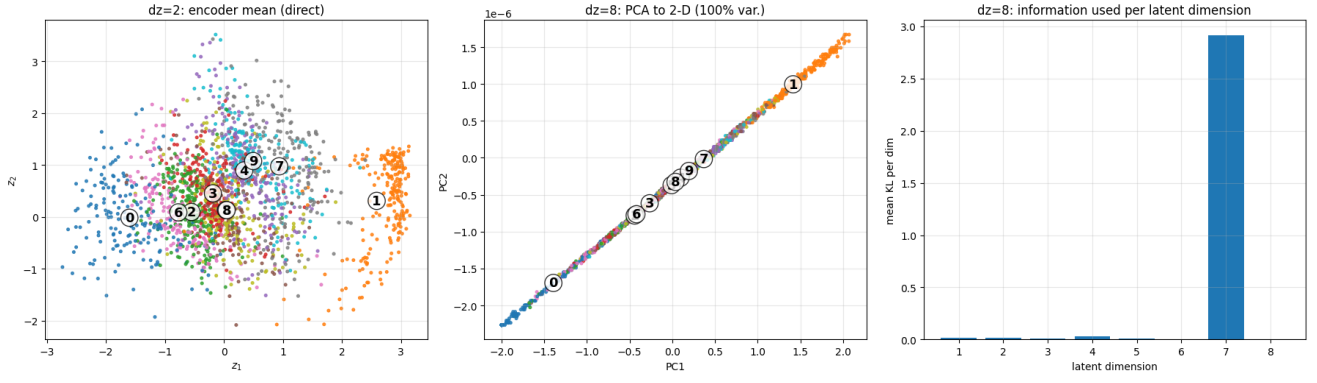


Figure 29: $d_z = 2$ latent space (left), $d_z = 8$ latent space projected to 2-D with PCA (centre), and the KL per latent dimension for $d_z = 8$ (right).

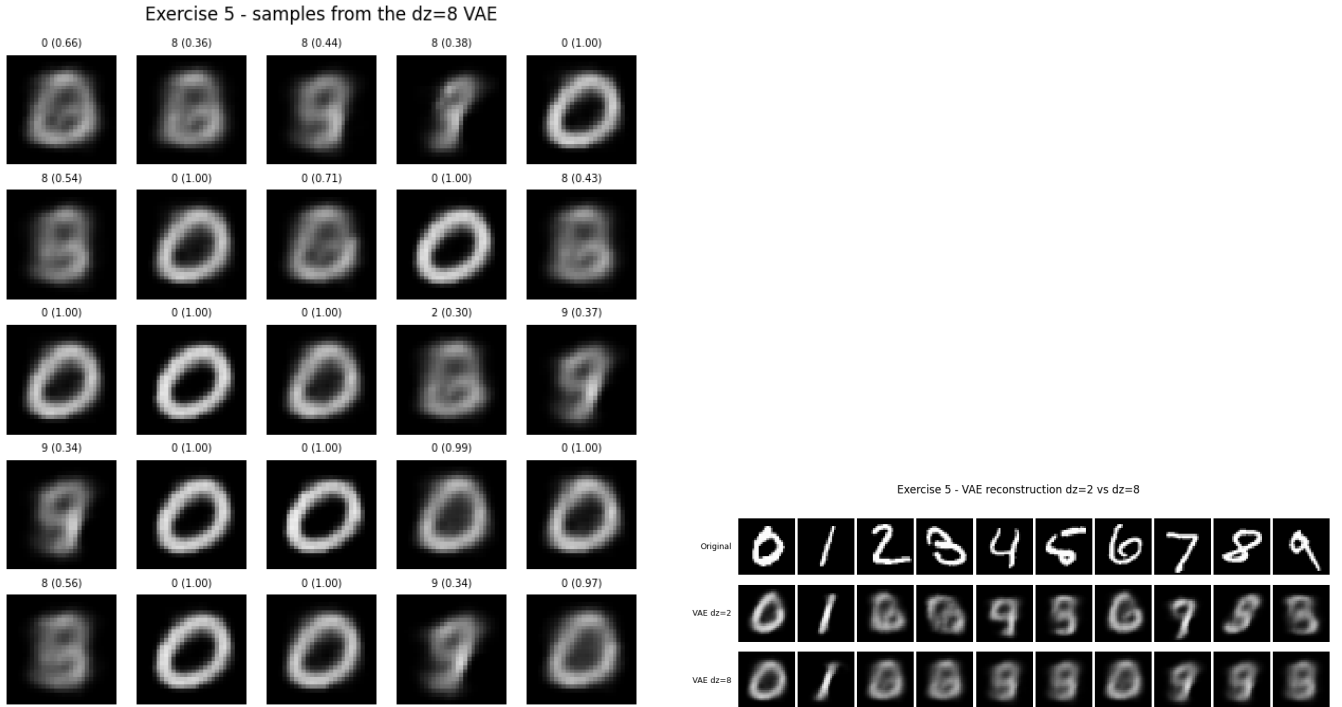


Figure 30: samples from the $d_z = 8$ VAE (left) and reconstructions of the $d_z = 2$ and $d_z = 8$ VAEs (right).

6. Investigate the effect of increasing the KL-loss contribution.

β	Rec. loss	KL loss	Total (Rec+ β KL)	MSE	SSIM	Latent std	5-NN acc. (%)
1	155.7024	5.2861	160.9884	0.0464	0.4414	1.0423	49.05
2	163.3589	4.6107	172.5803	0.0493	0.4109	1.0035	41.45
5	165.1326	2.7279	178.7720	0.0489	0.3990	0.9270	42.10
10	169.7254	1.7933	187.6585	0.0485	0.3866	0.8745	42.40

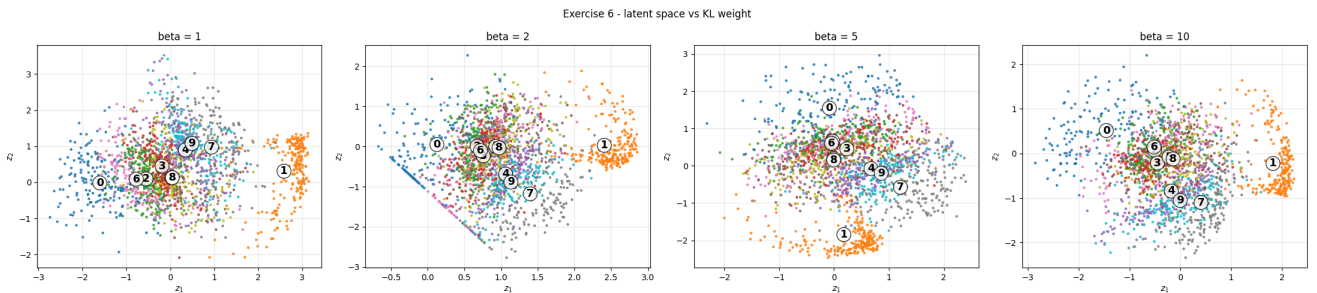


Figure 31: latent space for $\beta = 1, 2, 5, 10$.

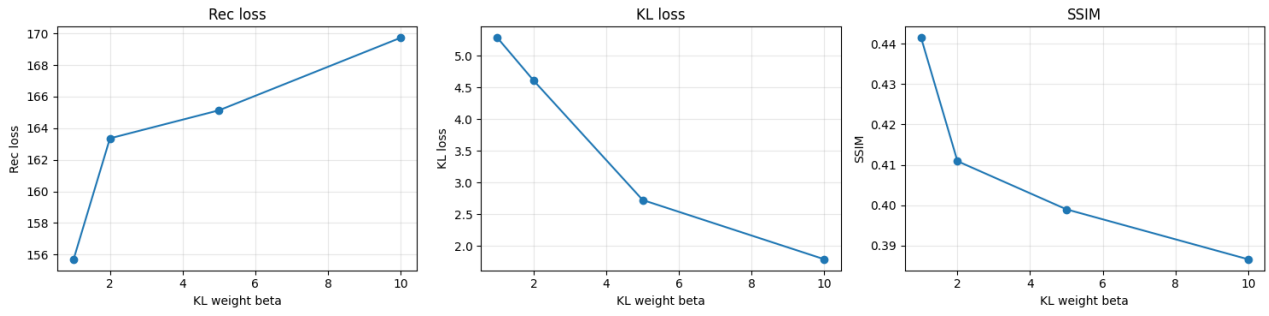


Figure 32: reconstruction loss, KL loss and SSIM versus the KL weight β .

Exercise 6 - samples for the same $z \sim \mathcal{N}(0, I)$ at different beta

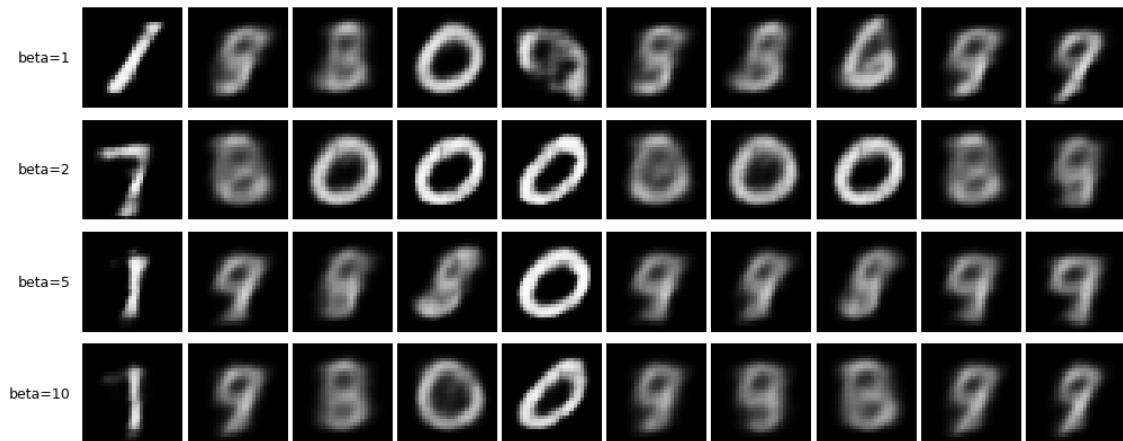


Figure 33: Exercise 6: samples decoded from the same $z \sim \mathcal{N}(0, I)$ for different β .

7. Generate a larger grid of VAE samples and discuss diversity.

Exercise 7 - 225 random VAE samples, $z \sim \mathcal{N}(0, I)$

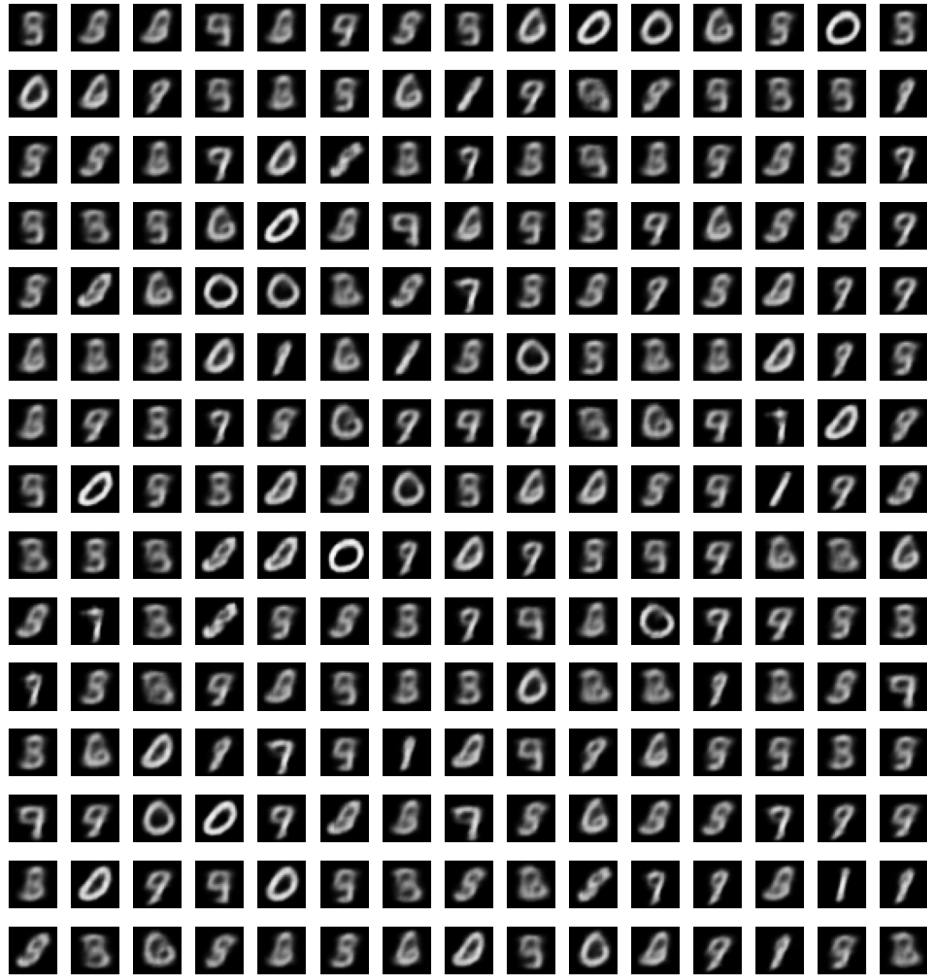


Figure 34: 225 random VAE samples, $z \sim \mathcal{N}(0, I)$.

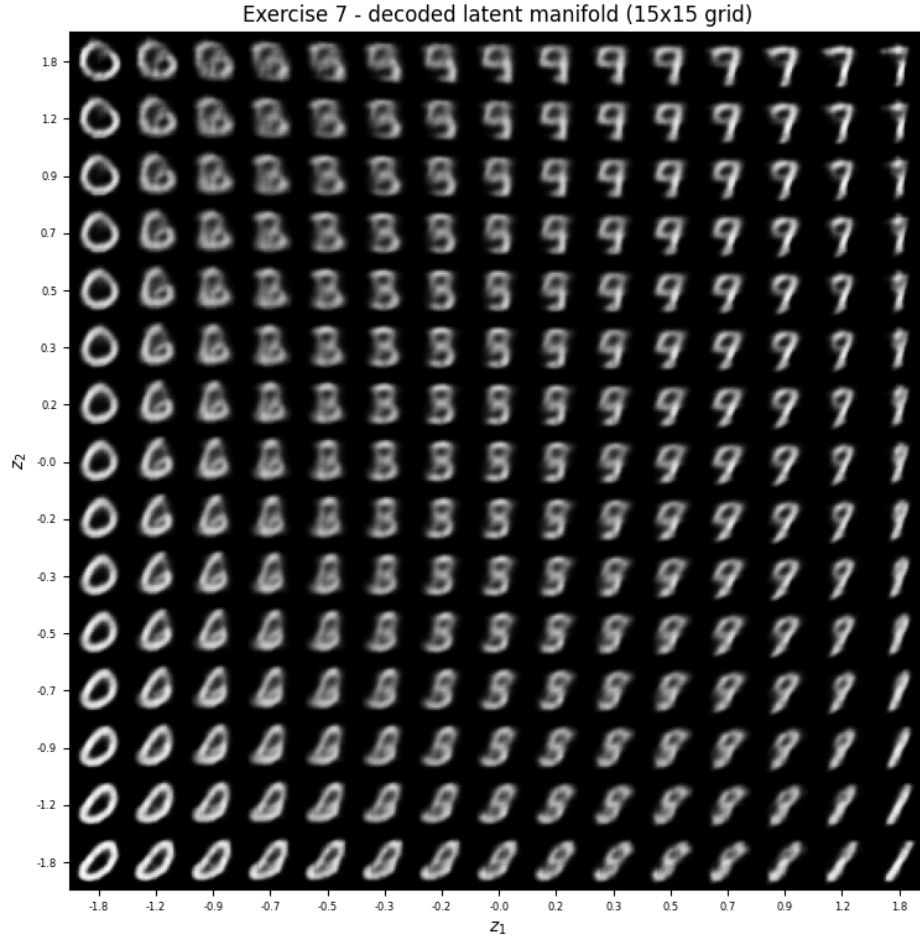


Figure 35: decoded latent manifold on a 15×15 grid of the two latent dimensions.

	Generated	Real test images
Class entropy (bits, max 3.32)	2.6666	3.3156
Mean pairwise pixel MSE	0.0365	0.1251
Mean classifier confidence	0.6488	0.9528

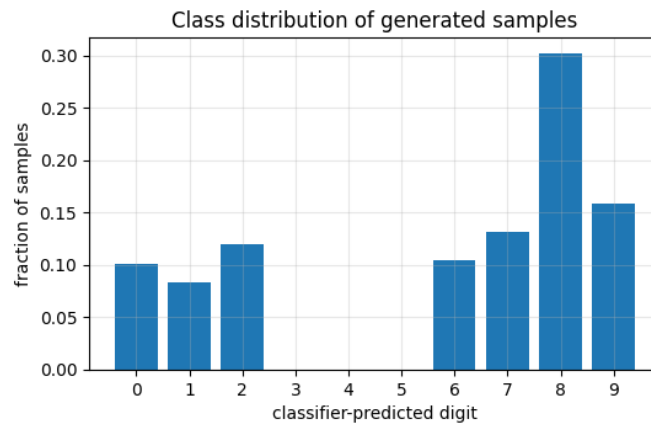


Figure 36: class distribution of 1,000 generated samples (classifier-predicted digit).

Path	Classifier predictions along the path
0→1	0, 6, 2, 8, 8, 9, 7, 7, 1, 1, 1
4→9	9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9
3→5	8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8
2→7	2, 8, 8, 8, 8, 9, 9, 9, 9, 7, 7

8. Select two digit regions in the latent space and perform interpolation.

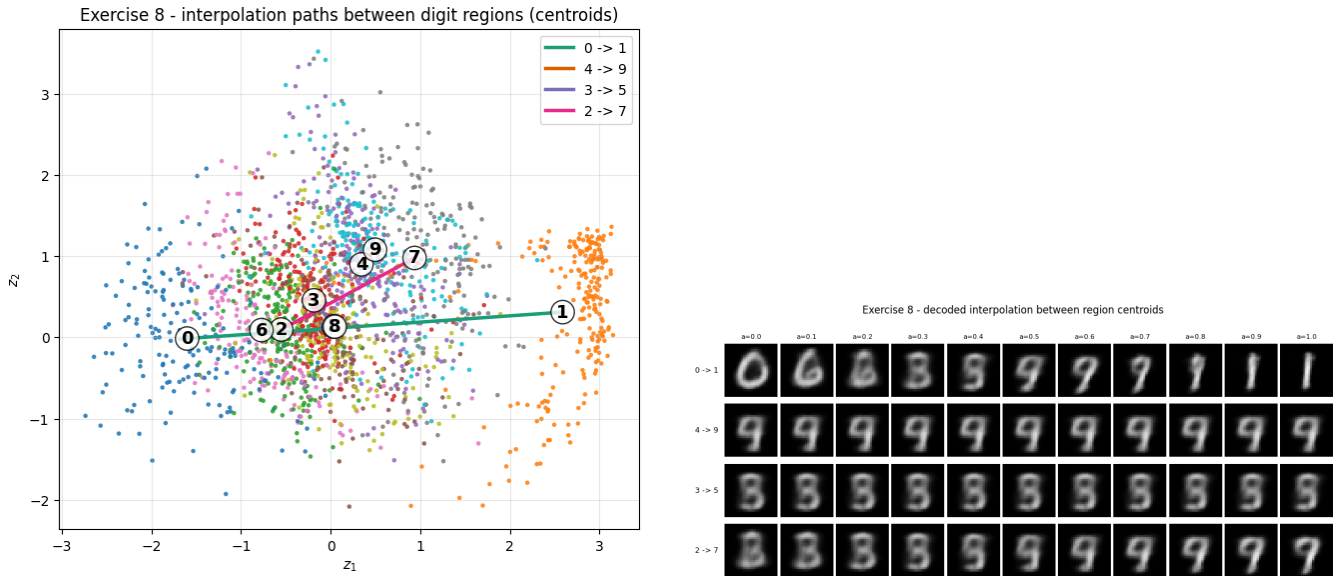


Figure 37: interpolation paths between the region centroids on the latent space (left) and the decoded images (right).

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>